

Subgraphs, Workflows & Multi-Agentic Systems

Session Notes — AI Engineering Course · Yash Jaiswar

Topics: LangGraph API · Graph vs Subgraph · Send API · 7 Workflow Patterns · Multi-Agent Architectures · Agent-as-Tool · Command API

❖ 1. LangGraph API — Complete Summary

Category	Methods	Purpose
Building a Graph	add_node, add_edges, add_conditional_edges, add_sequence, set_conditional_entry_point, set_finish_point	Define graph structure
Executing a Graph	invoke, stream, getState, getStateHistory, updateState	Run and inspect/modify at runtime
Controlling Flow	send — dynamic fan-out command (goto / update / resume) interrupt — pause for human input	Dynamic routing, HITL, multi-agent coordination

❖ 2. Graph vs Subgraph

Graph = nodes (functions) connected by edges (control flow).

Subgraph = a compiled graph added as a node inside a parent graph.

The label 'subgraph' only applies in context — both are just graphs.

Why use a subgraph?

- Isolation: each piece of logic lives in its own independently testable unit
- Maintainability: modifying a subgraph doesn't require touching the parent
- Reusability: the same subgraph can be called from multiple parent graphs
- Clarity: the parent graph stays lean — complexity is delegated downward

Real example from instructor: manufacturing system with installation, material, and packaging documents — each got its own subgraph. Fixing one didn't affect the others.

Scenario	Integration method
Same state schema as parent	Add compiled subgraph directly: <code>graph.add_node('name', subgraph)</code>
Different state schema	Wrap in a function that transforms state in/out; add that function as a node

■■ SESSION CORRECTION: Subgraph state keys DO NOT need to match the parent.

Each graph can define its own state schema independently.

Values that share a key name flow automatically; others stay scoped to their graph.

■■ COMMON MISTAKE: Subgraphs are NOT automatically about parallel execution.

The inventory subgraph (`check_inventory` → `calculate_price`) runs SEQUENTIALLY.

Parallel execution is a separate concern — it requires the Send API.

Subgraphs are about isolation and modularity first. Parallelism is optional.

❖ 3. Inventory Management — Subgraph Example

Component	What it does	Graph
OrderState	product, quantity, inventory_status, total_price, final_response	Shared
check_inventory	Checks stock vs requested quantity → sets inventory_status	Subgraph
calculate_price	If available: $\text{unit_price} \times \text{qty} = \text{total_price}$; else 0	Subgraph
validate_request	Validates product exists in catalog and $\text{qty} > 0$	Parent
order_processing	Node that hosts the compiled subgraph	Parent
create_final_response	Builds human-readable output from final state	Parent

■ How to add a compiled subgraph as a node:

```
order_subgraph = order_subgraph_builder.compile()
parent.add_node('order_processing', order_subgraph)
```

State values written by the subgraph (inventory_status, total_price) are automatically available in the parent graph after the subgraph runs.

❖ 4. Send API — Dynamic Parallel Dispatch

Send dispatches tasks to nodes/subgraphs **in parallel at runtime**.

The orchestrator decides how many workers to create DURING execution — not at compile time.

→ Each Send() call spawns an independent thread for that node/subgraph

→ Workers run simultaneously (fan-out)

→ Results collected via Annotated[list, operator.add] reducer (fan-in)

→ Send is always connected via a CONDITIONAL EDGE from the orchestrator node

	Regular Edge	Send API
Defined at	Compile time	Runtime — inside a node function
Branches	Fixed — same every run	Dynamic — decided during execution
Parallelism	No	Yes — multiple threads simultaneously
Connection	Normal or conditional edge	Always via conditional edge
Reducer needed	No	Yes — Annotated[list, operator.add]

Why conditional edge for Send? — A conditional edge routing function can RETURN Send() calls.

A normal edge just points to the next node — it has no return value and cannot dispatch Send.

The distribute_topics / routing function:

→ Loops over topics

→ Returns [Send('research_worker', {'topic': t}) for t in topics]

→ This is what triggers parallel threads — one per topic

→ LLM is NOT involved in this decision (unlike ReAct/tool-calling)

❖ 5. Research Agent — Send API Example

Component	Schema / Role
ParentState	topic: list, research_result: Annotated[list, op.add], final_answer: str
ResearchWorkerInput	topic: str (one subtopic — scoped to one worker)
ResearchState	topic, research, summary (scoped to subgraph only)
research_topic	LLM generates research paragraph for one topic (subgraph node 1)
summarize_topic	LLM summarizes the research (subgraph node 2)
research_subgraph	START → research_topic → summarize_topic → END
prepare_topic	Parent node — prints incoming topic list, passes state through
research_worker	Invokes research_subgraph for one topic, returns result
distribute_topics	Conditional edge fn — returns Send('research_worker', ...) per topic
combine_result	Receives full list, merges all results into final report

Full flow:

prepare_topic → distribute_topics [conditional edge]
→ [research_worker #1, #2, #3 ... in parallel via Send]
→ each invokes research_subgraph
→ combine_result → END

❖ 6. 7 LangGraph Workflow Patterns

6.1 Sequential

Steps are known and fixed. Nodes execute in one direction.

N1 → N2 → N3 → N4

Key mechanism: add_edges (normal edges only).

6.2 Fan-out / Fan-in (Parallel)

Fan-out: one node dispatches to N nodes simultaneously.

Fan-in: N nodes send results to one combiner node.

Use when: several independent jobs should all run at the same time.

Key mechanism: Send API + Annotated[list, operator.add]

Without reducer: parallel writes OVERWRITE each other — only last result survives.

With reducer: each result APPENDED — all N results collected.

6.3 Conditional Routing

Graph selects ONE branch based on state. Only one branch executes per run.

Key mechanism: add_conditional_edges + router function.

Example: tools_condition → ToolNode if tool_calls present, else END.

6.4 Orchestrator-Worker

Orchestrator decides AT RUNTIME which workers to spawn and what each receives.
Workers run in parallel via Send. Number of workers only known after orchestrator runs.

Key mechanism: Send API via conditional edge out of orchestrator node.
Workers can be a single node OR a compiled subgraph.
Fan-in: Annotated[list, operator.add] collects all worker outputs.

6.5 Feedback Loop — Evaluator-Optimizer

Generator produces output → Evaluator (LLM) scores it and writes feedback to state.
If score is poor, graph loops back to Generator WITH the feedback attached.
Generator retries using the evaluator's critique — NOT a blind retry.

START → Generator → Evaluator
↓ pass → END
↓ fail → feedback added to state → Generator (retry)

Key mechanism: conditional edge from Evaluator back to Generator.
ALWAYS set a max attempt limit to prevent infinite loops.

Distinction from basic retry loop: the FEEDBACK IN STATE is what makes it
Evaluator-Optimizer. A basic retry just re-runs — no critique, no improvement signal.

6.6 Tool Calling / ReAct Loop

LLM itself decides which tool to call. Tool is bound to the model — not the graph.
LLM → (has tool_calls?) → ToolNode → result back to LLM → repeat or END
Key mechanism: bind_tools + tools_condition conditional edge.
The MODEL controls tool selection — not the graph structure.

6.7 Agent-as-Tool

Each 'tool' in a React agent is itself another React agent.
Outer LLM decides which sub-agent to call via its own ReAct reasoning loop.

Example: main_coordinator (LLM) has tools: ask_research_agent, ask_math_agent, ask_writing_agent.
Each tool = create_agent(model, tools, prompt) — a full React subagent.

Key mechanism: create_agent (LangChain) wraps a React agent as a callable.
The entire system is still ONE React agent — outer LLM + its sub-agent tools.
No separate supervisor node exists in the graph.

Pattern	Flow	Key mechanism	Dynamic?
Sequential	Fixed sequence	add_edges	No
Fan-out / Fan-in	Parallel (fixed N)	Send + operator.add	Yes

Routing	One conditional branch	add_conditional_edges	Yes
Orchestrator-Worker	Dynamic parallel fan-out	Send via conditional edge	Yes
Evaluator-Optimizer	Feedback loop with critique	Conditional edge + feedback state	Yes
ReAct / Tool Calling	Model-driven tool decisions	bind_tools + tools_condition	Yes
Agent-as-Tool	Nested React agents as tools	create_agent + outer LLM	Yes

◆ 7. Multi-Agent Systems

Definition: More than one agent working together, collaborating toward a shared goal.

Agent = a system running in dynamic/autonomous mode — LLM decides at runtime.

Deterministic flow: steps predefined, order fixed → Sequential, routing patterns

Dynamic flow: decisions made at runtime → achieves autonomy → ReAct, Orchestrator-Worker

Partial Agentic System: LLM routing + fixed LLM nodes downstream.

Not fully autonomous — but handles 90% of real-world requirements.

7.1 Three Multi-Agent Architecture Patterns

Pattern	Structure	When to use
Supervisor	Central supervisor LLM → delegates to specialist agents	Most common — structured, predictable
Network	Agents connect directly to each other, no supervisor	Peer-to-peer, independent collaboration
Hierarchical	Supervisor → sub-supervisors → sub-agents (tree)	Complex multi-level tasks

7.2 Agent-as-Tool vs Supervisor — Key Distinction

Dimension	Agent-as-Tool	Supervisor Multi-Agent
Graph structure	One React agent — no supervisor node	Separate supervisor node in the graph
Control mechanism	Outer LLM's ReAct loop decides which sub-agent to call	Supervisor LLM routes via conditional edges to specialist nodes
Sub-agents	Wrapped as tools via create_agent()	Separate nodes/subgraphs in the graph
Looks like	Similar output on surface	Similar output on surface
Underlying pattern	Single React agent (one node calling tools)	Multi-node graph with explicit routing logic

7.3 When to Use Supervisor Multi-Agent (not just routing)

Introduce a supervisor multi-agent system **ONLY** when:

- Each sub-agent needs its own tools, reasoning loop, and independent context
- The routing between agents cannot be hardcoded — it requires dynamic LLM judgment at runtime
- You have evaluated evidence that a simpler conditional-edge approach genuinely fails
- The system is well-tested before going to production

In practice: supervisor flows have less chance of success because more autonomy = more cost + more failure surface. Start simple. Add complexity only when forced to.

7.4 Deep Agents

Term	Definition
Deep Agent	Agent running for long horizon — many reasoning steps, complex multi-step planning
Can be	Single agent OR multi-agent — either can be deep if it runs long enough
Cost warning	Frequent LLM API calls = expensive. Test thoroughly before productionizing.
Examples	GitHub Copilot Agent, Claude Code, Cursor — all deep agents

❖ 8. Command API

Sub-function	What it does	New or seen before?
goto	Routes execution to a specific node programmatically	NEW — not used in any practical yet
update	Modifies graph state directly from inside a node	NEW — not used in any practical yet
resume	Resumes graph from interrupt() checkpoint	SEEN — Challenge 4: Command(resume=...) in HITL

Command API gives programmatic control over flow — beyond what edges can express.
Used mainly inside multi-agentic flows where routing logic is too complex for edges alone.

resume = same Command(resume=...) from Challenge 4 HITL practical.
goto and update are new capabilities you haven't built with yet.

❖ 9. Corrections from This Session's Review

■ Q1 — Subgraph reason: 'to enable parallel execution via fan-out'

CORRECTION: The inventory subgraph runs SEQUENTIALLY.
Subgraphs are about ISOLATION and MODULARITY — not parallel execution.
Parallel execution is separate and requires Send API.
The real benefit: each piece of logic is independently testable and modifiable.
Changing the inventory logic doesn't require touching the parent graph.

■ Q2 — Conditional edge in Orchestrator-Worker: 'LLM decides which node to use'

CORRECTION: No LLM is involved. That describes the ReAct/tool-calling pattern.
In Orchestrator-Worker, the conditional edge routing function RETURNS Send() calls.
A normal edge has no return value — it cannot dispatch Send.
The conditional edge is structural: it's the only edge type that allows Send() dispatch.

■ Q3 — Evaluator-Optimizer vs retry loop: missed the key distinction

CORRECTION: The defining feature is FEEDBACK IN STATE.

A basic retry loop just re-runs the generator — no critique, no improvement signal.

Evaluator-Optimizer: the evaluator (LLM) writes critique/feedback into the state.

The generator retries WITH that feedback — it knows what to improve.

Also missed: max attempt limit is required to prevent infinite loops.

■ Q3 (Command API) — 'goto AND resume overlap with previous practicals'

CORRECTION: Only RESUME overlaps. You used Command(resume=...) in Challenge 4 (HITL).

goto routes execution to a specific node — not used in any practical yet.

update modifies state directly — not used in any practical yet.

◆ 10. Coming Next

Session	Topic
Next	MCP (Model Context Protocol) + A2A (Agent-to-Agent protocol)
After that	Guardrails
Class 50–51	Evaluation (DeepEval framework)
Final	End-to-end Projects + Azure Cloud Deployment